Report

Name: Levente Olivér Bódi

Student Number: s1122062

Implementation

I implemented the code with two classes, the logger and the buffer classes, in a way that multiple instances of the buffers can share the logger.

The logger stores strings as entries in a vector. It uses a shared mutex lock so that multiple buffers can read it (an entry, all entries or the size of it), while writers obtain a unique mutex lock for writing. Readers return single entries, the size, or copies of the entire log making it non-destructive.

Each instance of a buffer manages a vector of integers and protects all internal state with a mutex. The buffer class supports the addition of an integer number at the back of the buffer if the buffer is unbounded or if the addition does not exceed the bound, otherwise it fails. It also supports a removal of an integer from the front of the buffer, if it is not empty, otherwise it fails. It supports to setting the bound size, if it is unbounded, or change it, if it has already been assigned a bound in a way that if the already existing size of the buffer instance is larger than the assigned bound, it keeps those integers, and fails the addition of integers until the size of the buffer does not drop under the assigned bound by calling multiple remove()'s. It supports making the buffer unbounded as well if it has been bounded.

All operations on the buffer (fail or success) are logged in the shared logger. It is done by incrementing seq_ under the mutex lock, releasing it, and calling the logger to record a line with operation details.

Concurrent usage

Each buffer has its own mutex, which protects the internal state (buf_, bounded_, bound_, seq_) of the buffer instance. Threads that operate on different buffers never race for a common lock, so they can be fully run in parallel. The add()/remove()/set_bound()/make_unbounded() operations take the mutex lock of the buffer only long enough to check its preconditions, mutate its state, and increment seq_ per buffer, ensuring minimal critical sections inside the buffer. Besides, all mutations on formatting and logging is done afterwards, which makes the region which threads race for minimal and maximize overlap among threads accessing the same buffer. Another point is that there are no nested locks inside the buffer implementation, which means that a buffer cannot block another one while logging for example.

Race conditions

My code does not suffer from race conditions because of the behaviour mentioned above, by concurrent usage. Each buffer instance's internal state is protected by a mutex lock. The internal states can be modified by add()/remove()/set_bound()/make_unbounded() if they obtain this lock. Besides, the check and mutation on seq_ is done atomically with respect to other threads. On the logger, there is concurrency but only reading; it is protected by an unique mutex lock for writing, and a shared for reading. There are also no nested locks in the code.

Deadlocks

By having no nested locks, we cannot have cycles. Besides, every buffer operation holds one lock at a time, so there is no such case that two operations wait for one another's lock while holding the lock which the other operation is waiting for. The operations are non-blocking by design, meaning that add() and remove() fail fast when the buffer is full or empty accordingly instead of waiting. The logger is also deadlock-free by design, it waits for a mutex lock for short pushback when a buffer writes to it, and has a shared mutex lock when buffers try to read from it.

Tests

I have completed 9 tests, from T1 to T9.

T1 checks the single threaded functional correctness of my implementation by trying out remove() on empty, several add(), several remove(), size(), and log length functions. I conducted it because it is necessary to ensure the baseline semantics before adding concurrency. It catches basic logical errors.

T2 checks the bounded capacity, meaning, that if the size let's say 3 and we try to push a 4th integer, it will fail. It verifies that the buffer boundedness works and that add() results in a fail when trying to add more items than the bound.

T3 tests whether policies can be switched during runtime. It starts with an unbounded buffer, then sets the bound to 3, verifies that add() fails when buffer is full, it removes integers below drop size, so that add succeeds. It is necessary because it checks whether the bound can be modified dynamically during runtime.

T4 tests 1 producer and 1 consumer working on the same buffer. When those two are working concurrently with add() and remove() operations, it checks whether they do not exceed the bound. It is useful to test concurrent stress on the same buffer to see that no error occurs while the buffer remains functionally correct.

T5 tests the logger, namely, when multiple buffers share it. It checks whether the two buffers working on the logger appear, and early lines show interleaving. It is necessary to check that the logger supports multiple writers without compromising the entries.

T6 tests the logger's concurrent reads. While a writer is working on the log, two readers call logger->size() and read_all(). This validates the reader-writers locking, also paying attention to possible deadlocks.

T7 performs a sequence of operations on one buffer, then verifies seq_ per buffer, if it does not have any gaps and runs from 1 to N. It confirms that the increment is correct and checks atomicity.

T8 forces remove() on an empty buffer, which still should fail, fills a small bounded buffer, which also should fail, and also provokes multiple remove()'s on an unbounded buffer, expecting it to fail. It is similar to T2. It ensures that failures only happen for the right reasons and that these are logged.

T9 lastly checks whether reading from the logger does not compromise its entries. It matches the specification that if an entry or the entire log has been read, the entries should still remain in the log.

Regarding the results, I built a TestResult struct, which I later on use in the expect(testresult, condition, message) function. The message parameter indicates in words what I expect, so it is in the code for each function.

From the tests it seems that my implementation does it correctly what it should do, however we can never be fully overarching with tests as discussed in my previous report/ assignment.

Test results print:

Summary:

T1: Single-thread functional correctness: 8 passed, 0 failed

T2: Bounded capacity enforcement: 5 passed, 0 failed

T3: Runtime policy switch: 10 passed, 0 failed

T4: 1P/1C same buffer: 3 passed, 0 failed

T5: Shared logger interleaving: 2 passed, 0 failed

T6: Logger concurrent reads: 1 passed, 0 failed

T7: Sequence linearization: 2 passed, 0 failed

T8: Failure semantics: 3 passed, 0 failed

T9: Logger read non-destructive: 2 passed, 0 failed

TOTAL: 36 passed, 0 failed